



WeCloudData

Debugging and Controlling Agents

Introduction





WeCloudData

Why Are Agents Difficult to Debug?

Debugging and Controlling Agents



Why Are Agents Hard to Debug?

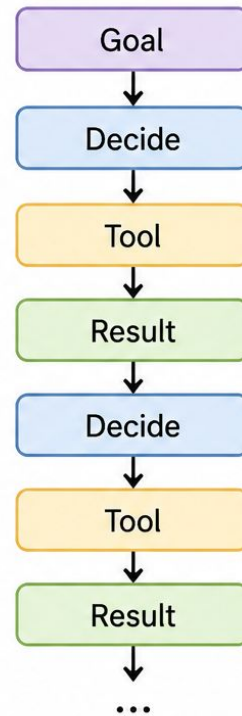
Debugging Agents

A traditional Application



More steps = more places to fail.

An Agent



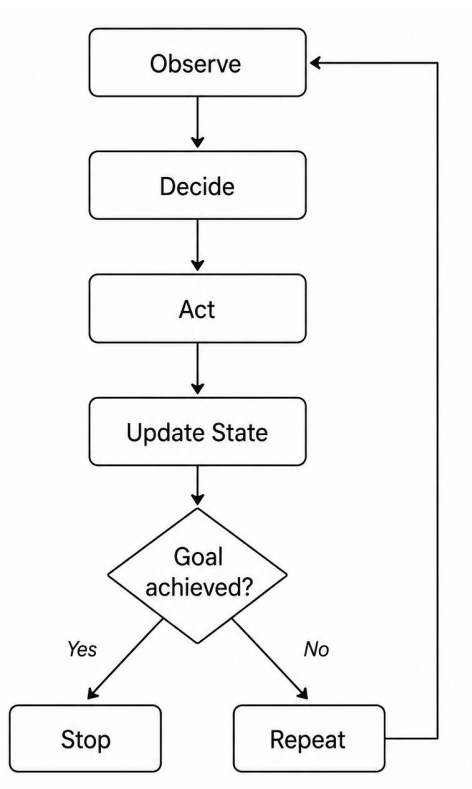


The Agent Loop

Debugging Agents

A Simple Mental Model:

This loop is the foundation for debugging agents.

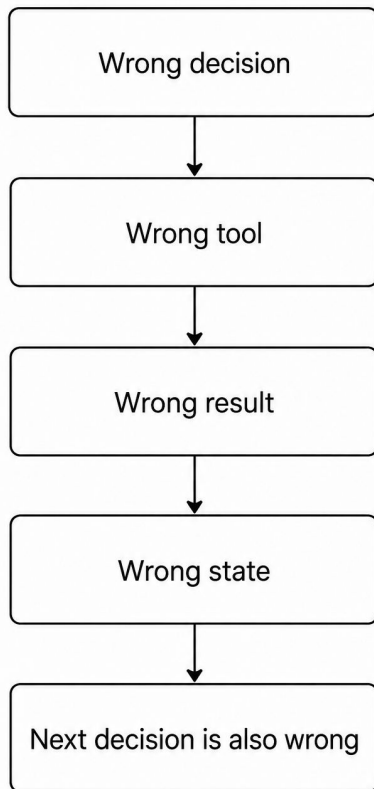




Why Failures Propagate?

Debugging Agents

Imagine:



An early mistake can affect everything that follows.



Example: Calendar Agent

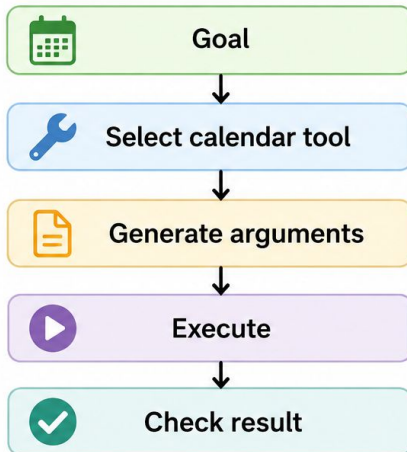
Debugging Agents



User

“Schedule a 30-minute meeting with Sarah tomorrow at 3 PM.”

Agent:



But what if the agent generates:

end_time = 2:30 PM



Incorrect meeting time!

The meeting will be 1 hour early instead of at 3 PM.



What Can We Actually Observe?

Debugging Agents

We may observe:

- Model inputs/outputs or supported summaries
- Tool calls
- Tool arguments
- Tool results
- Errors
- State changes
- Retries
- Guardrail decisions
- Agent handoffs
- Timing, tokens and cost
- Termination events

Observability is not the same as seeing the model's complete hidden reasoning.



Why We Need Tracing?

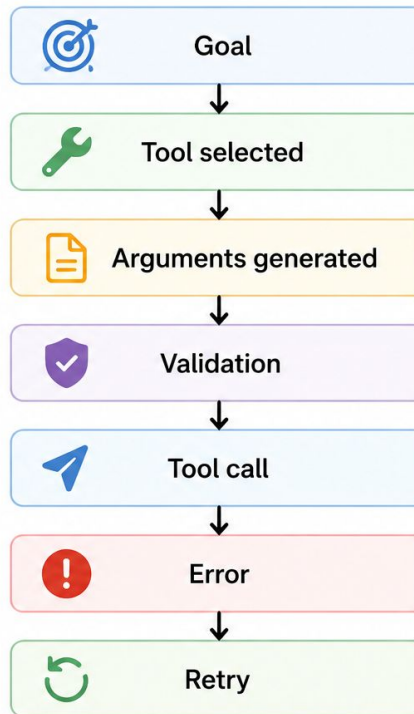
Debugging Agents

1. Normal application log:

`Calendar API called`

✗ Not enough.

2. Agent trace:





WeCloudData

What an Execution Trace Captures? Debugging and Controlling Agents

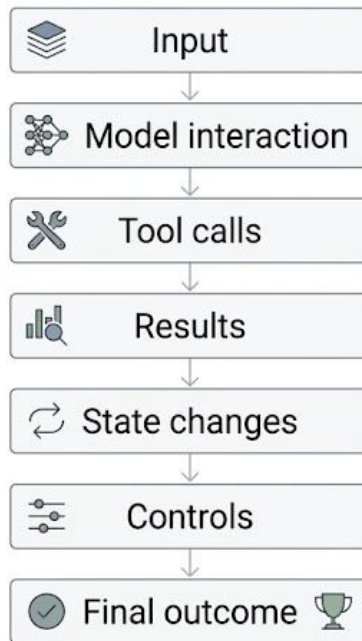


What Is an Execution Trace?

Execution Trace

An execution trace is a **structured record of what happened during an agent run.**

It connects:





Trace $\neq$ Hidden Reasoning

Execution Trace

Don't Think:

"The trace shows exactly what the model was thinking."

Think:

"The trace shows the observable events produced during execution."

Example:

Tool selected:
 `create_calendar_event`

Arguments:
 `start_time = ...`

Result:
 `PERMISSION_DENIED`



What Does a Trace Contain?

Execution Trace

Model interaction

- Request
- Response / supported summary

Tool execution

- Tool name
- Arguments
- Result
- Error

System behavior

- State changes
- Retries
- Guardrails
- Approvals
- Handoffs



A Trace Has a Timeline

Execution Trace

Why timestamps matter?

They help identify:

- Slow steps
- Repeated calls
- Delays
- Cascading failures

09:01:02 User request
09:01:03 Tool selected
09:01:03 Arguments generated
09:01:03 Validation failed
09:01:04 Arguments corrected
09:01:05 Tool executed
09:01:05 Success



Tool Events

Execution Trace

For each tool call, capture:

Tool: create_calendar_event

Arguments: {...}

Result: {...}

Status: success / failure

Latency: ...

Retry: yes / no

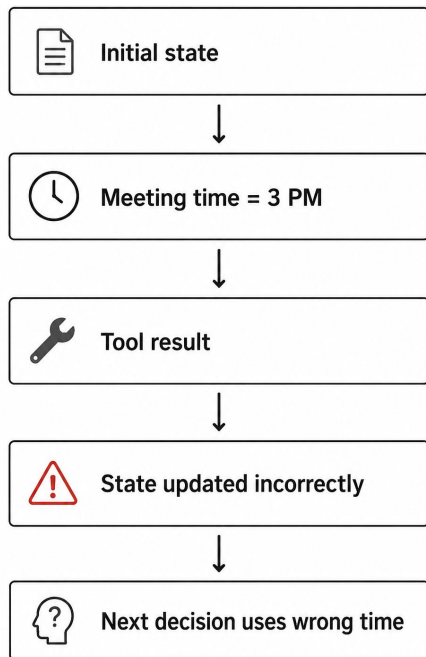
This makes tool failures much easier to diagnose.



State & Memory Events

Execution Trace

Agents can also fail because state changes incorrectly.



Trace the state transitions.



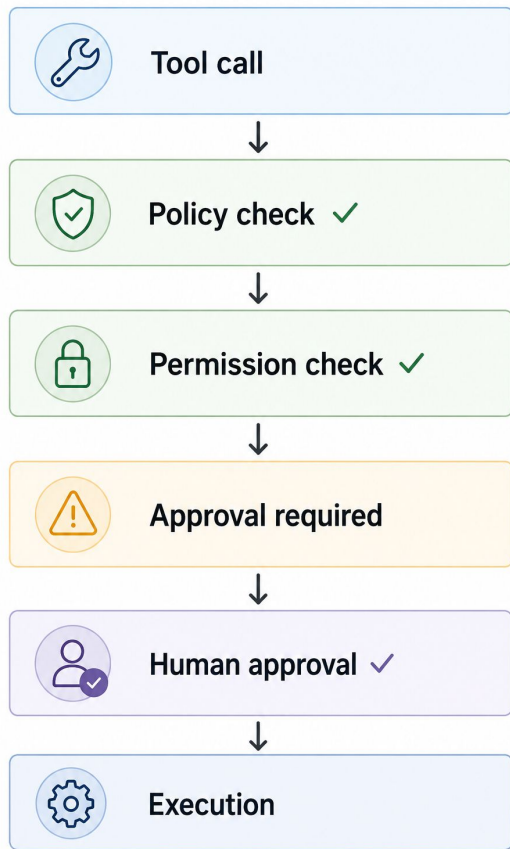
Controls in the Trace

Execution Trace

A useful trace can show:

Why?

Controls are part of the execution, not an afterthought.





Don't Log Everything

Execution Trace

More logs $\neq$ better debugging.

Sensitive information may include:

- User identifiers
- Personal information
- Prompts
- Tool outputs
- API responses

Good practice

Collect what you need, minimize sensitive data, redact where appropriate, and control access and retention.



From Logs to Traces

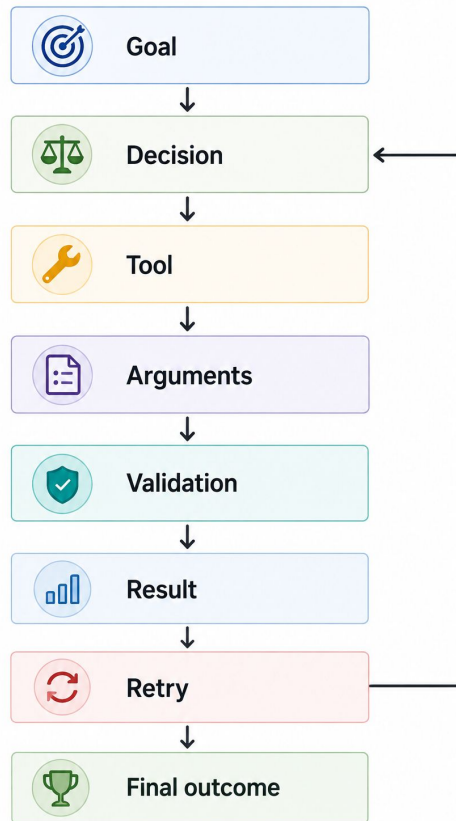
Execution Trace

Traditional logs

API called
Error occurred

Trace = context + sequence + relationships

Agent trace





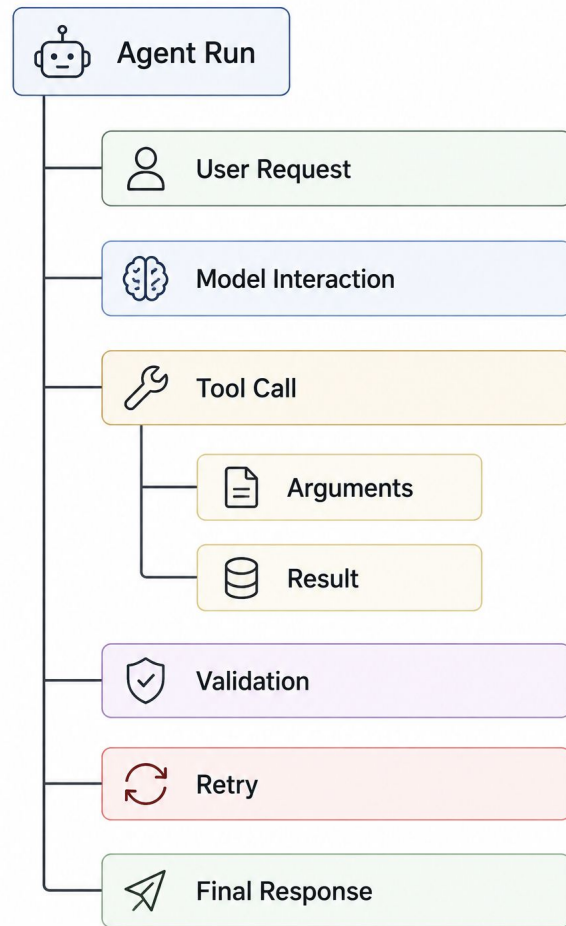
Trace Visualization

Execution Trace

A useful trace might look like:

Goal:

See the execution path, not just individual log messages.





WeCloudData

A Practical Debugging Workflow

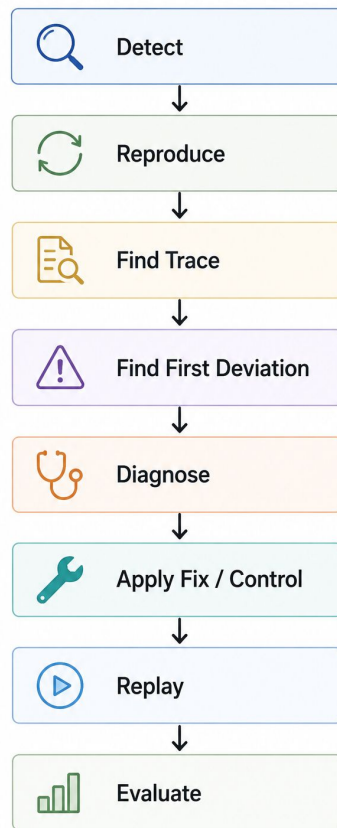
Debugging and Controlling Agents



Debugging an Agent

Debugging Workflows

When something goes wrong:





Step 1: Reproduce the Failure

Debugging Workflows

Start with

What happened?

Capture

- User input
- Agent configuration
- Tools available
- Relevant state
- Model/version
- Trace

Goal:

Make the failure observable and repeatable.



Step 2: Find the Trace

Debugging Workflows

Don't start by reading every event.

Start with

Which run failed?

Capture

- First error
- Unexpected tool call
- Invalid arguments
- Unexpected state change
- Retry loop
- Policy violation

Goal:

Locate the failed run and its first useful signal.



Step 3: Find the First Meaningful Deviation

Debugging Workflows

Suppose:

- Tool selection ✓
- Arguments ✓
- Validation ✓
- Execution ✓
- State update ✗
- Next decision ✗
- Final response ✗

Root cause

The first meaningful deviation may be:

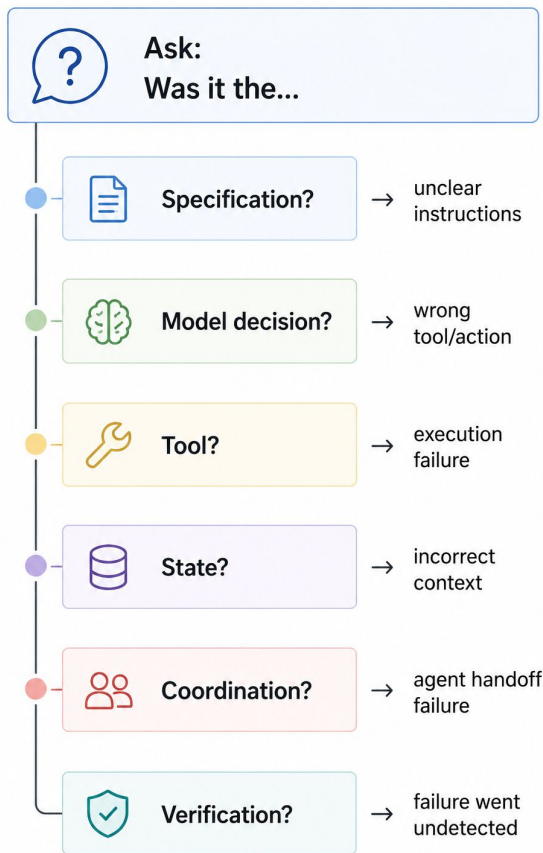
Incorrect state update

Not the final incorrect response.



Step 4: Identify the Failure Layer

Debugging Workflows





Step 5: Fix the Right Layer

Debugging Workflows

Don't automatically change the prompt.

Example

Wrong tool selected



Could be:

- unclear tool description
- poor tool selection logic
- Missing routing constraint

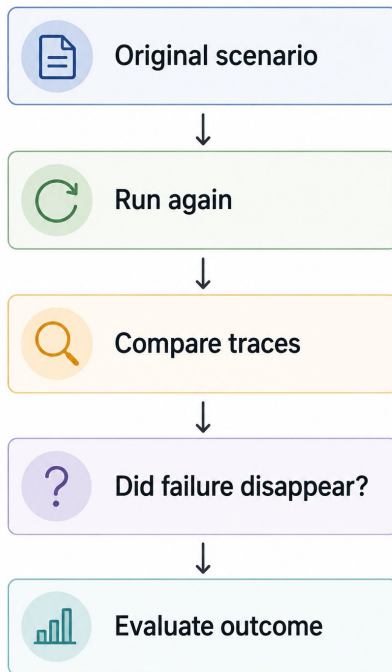
Fix the root cause, not the symptom.



Step 6: Replay the Scenario

Debugging Workflow

After applying a fix:



A fix isn't complete until it is tested.



Debugging Walkthrough

Debugging Workflows

User

"Schedule a meeting
with Sarah tomorrow at
3 PM."

Trace

- Tool selected ✓
- Arguments generated ✓
- Validation ✓
- Permission ✓
- Tool execution ✓
- Result = conflict
- Retry...

What looks suspicious?

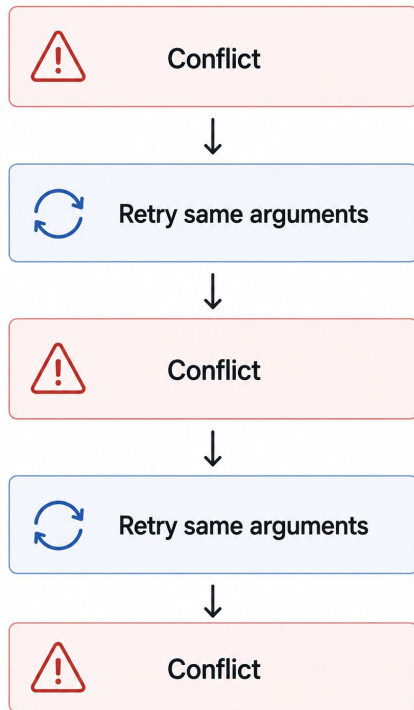
The retry behavior.



Find the First Deviation

Debugging Workflows

Trace shows:



Diagnosis

The agent is retrying
**without changing the
condition causing
failure.**



Apply a Control

Debugging Workflows

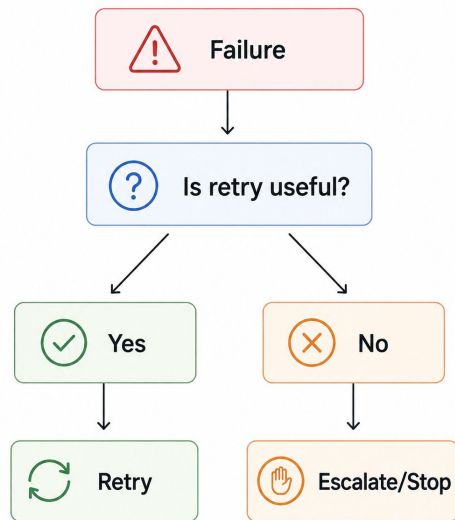
Instead of:

Failure → Retry

Add:

Maximum retry limit

Use





WeCloudData

Failure Modes as Diagnostic Patterns

Debugging and Controlling Agents



Agent Failure Taxonomy

Diagnostic Patterns

Most failures fall into a few patterns:

- Specification
- Decision
- Context / State
- Tool Execution
- Coordination
- Verification
- Loops / Cascades

For every failure ask:

What happened? → What does the trace show? → What control prevents it?



Specification Failure

Diagnostic Patterns

What happened?

The agent follows an unclear or conflicting instruction.

Trace evidence

Ambiguous task
↓
Unexpected action

Control

- Clear instructions
- Explicit constraints
- Better tool descriptions
- Validation



Decision Failure

Diagnostic Patterns

What happened?

The agent chooses the wrong action.

Example: User asks to create an event.

Agent:

`search_calendar()`

Trace evidence

User goal



Wrong tool selected

Control

- Better tool contracts
- Routing constraints
- Tool-selection evaluation



Context / State Failure

Diagnostic Patterns

What happened?

The agent loses or corrupts information needed for later steps.

Example:

Meeting = 3 PM
↓
State update
↓
Meeting = 2 PM ✗

Control

- Explicit state representation
- State validation
- Trace state transitions



Tool Execution Failure

Diagnostic Patterns

What happened?

The agent makes a reasonable call, but the tool fails.

Examples:

- API unavailable
- Timeout
- Permission denied
- Invalid external data

Trace evidence

Tool call ✓
Execution ✗
Error = TIMEOUT

Control

Retry policies +
fallback + clear
error handling



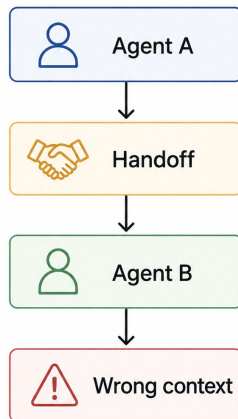
Coordination Failure

Diagnostic Patterns

What happened?

Especially important in multi-agent systems.

Examples:



Trace evidence

- Incorrect handoff
- Missing context
- Conflicting state

Control

- Explicit handoff contracts
- Shared state rules
- Validation between agents



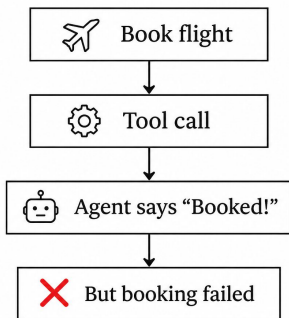
Verification Failure

Diagnostic Patterns

What happened?

The agent performs an action but never checks whether it actually succeeded.

Examples:



Control

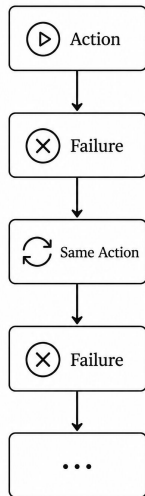
- Verify the outcome before claiming success.



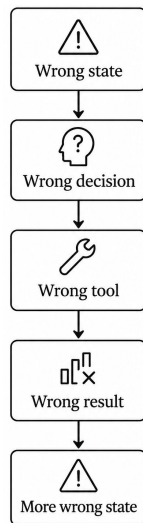
Looping & Cascading Failures

Diagnostic Patterns

Loop



Cascade



Control

- Retry limits
- State checks
- Progress detection
- Stopping conditions



Looping & Cascading Failures

Diagnostic Patterns

Research Agent



Planning Agent



Execution Agent

A failure may originate from:

- Wrong handoff
- Missing context
- Conflicting instructions
- Incorrect ownership
- Bad shared state

Debug the **handoff**, not just each agent.



Failure → Evidence → Control

Diagnostic Patterns

Failure	Trace Evidence	Control
Wrong tool	Incorrect tool call	Better tool contract
Invalid arguments	Validation error	Schema + semantic validation
State corruption	Unexpected state change	State checks
Tool failure	API error	Retry/fallback
Verification gap	No success confirmation	Outcome verification
Loop	Repeated same action	Retry/turn limits
Risky action	Policy violation	Permission/approval



WeCloudData

Bounded Autonomy as Layered Control **Debugging and Controlling Agents**



What Is Bounded Autonomy?

Bounded Autonomy

An agent can act independently but **within defined boundaries**.

$$\begin{array}{c} \text{Autonomy} \\ + \\ \text{Boundaries} \\ = \\ \text{Controlled Agent} \end{array}$$

Why?

Because not every action should be equally autonomous.

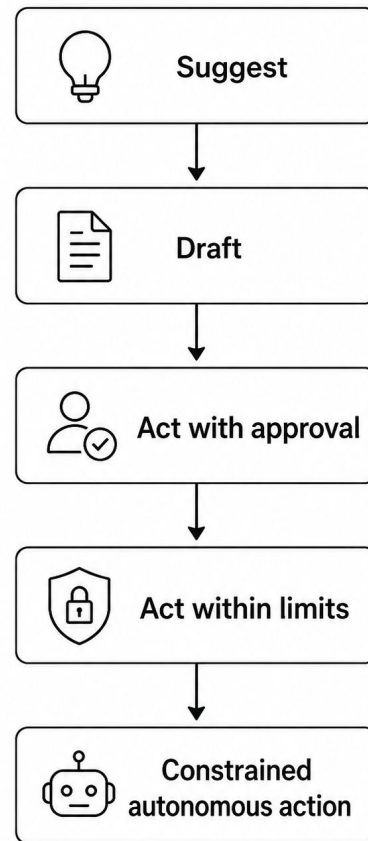


Autonomy Is a Continuum

Bounded Autonomy

An agent can act independently
but **within defined boundaries**.

This is a **practical continuum**,
not a universal formal standard.





Permission Boundaries

Bounded Autonomy

Not all tools have the same risk.

Read

- Search calendar
- View account

Write

- Create event
- Send email
- Update record

Irreversible / high impact

- Delete data
- Make purchase
- Transfer funds

Higher-risk actions need stronger controls.



Approval Based on Risk

Bounded Autonomy

Not all tools have the same risk.

Low risk



Automatic

Medium risk



Approval if needed

High risk



Human approval

Examples:

- Search flights → automatic
- Book flight → approval
- Purchase expensive ticket → explicit approval



Human-in-the-Loop

Bounded Autonomy

Not all tools have the same risk.

Before

"Do you want me to book this?"

During

"The action requires approval."

After

"The system completed the action; please review."

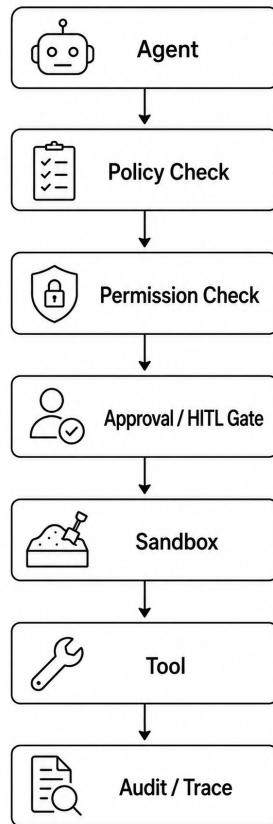
Human involvement should happen at meaningful decision points.



Layered Control Architecture

Bounded Autonomy

Don't rely on the prompt alone.





Prompt vs System Controls

Bounded Autonomy

Not all tools have the same risk.

Prompt instruction

"Do not delete
files."

Helpful—but not sufficient.

System enforcement

Delete request



Permission check



BLOCKED

High-risk controls should be enforced outside the model whenever possible.



WeCloudData

Stopping, Pausing & Escalating Debugging and Controlling Agents



Agents Don't Have One "Stop"

Stopping, Pausing and Escalating

An agent may:

- Complete successfully
- Fail safely
- Pause for approval
- Escalate to human
- Terminate due to limits

Stopping is a design decision.



Hard Limits

Stopping, Pausing and Escalating

Define boundaries such as:

- Maximum turns
- Maximum retries
- Maximum execution time
- Maximum token usage
- Maximum cost

Example

Maximum attempts = 3

1 → retry

2 → retry

3 → stop

Hard limits should be enforced by the system.



Outcome-Based Stopping

Stopping, Pausing and Escalating

Don't stop only because you've reached a number.

Also stop when:

Success

Required
outcome verified

Risk

Policy violation
detected

Uncertainty

Result cannot be
reliably verified

Approval

Human decision
required



Confidence Isn't Always Enough

Stopping, Pausing and Escalating

The model might say:

"I'm 95% confident this is correct."

But:

Self-reported confidence is not necessarily calibrated.

For high-risk decisions, prefer:

- External verification
- Deterministic rules
- Evaluation signals
- Explicit approval



Prompt Stop vs Hard Stop

Stopping, Pausing and Escalating

Prompt-level

"Stop after three attempts."

Useful guidance.

Orchestration-level

```
if retries >= 3:  
    stop()
```

Better:

Use the model for decisions, but enforce critical limits outside the model.



When Should We Escalate?

Stopping, Pausing and Escalating

Escalate when:

- User intent is ambiguous
- Permission is required
- Risk is high
- Tool results are contradictory
- Verification fails
- Repeated attempts don't work
- The system reaches a predefined limit



Premature vs Delayed Stopping

Stopping, Pausing and Escalating

Too Early

Temporary error



STOP

Potentially unnecessary failure.

Too Late

Failure



Retry



Retry

Potentially dangerous.

Goal:

Stop when continuing is no longer useful, safe, or authorized.



Stopping Checklist

Stopping, Pausing and Escalating

Before deployment, define:

- Maximum turns
- Maximum retries
- Time limit
- Cost limit
- High-risk actions
- Approval points
- Verification requirements
- Escalation conditions
- Safe failure behavior



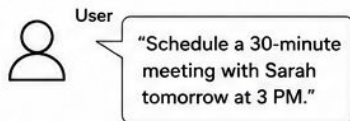
WeCloudData

End-to-End Controlled Agent Debugging and Controlling Agents

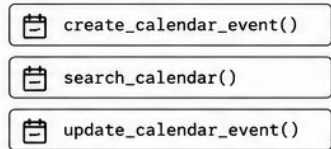


End to End Controlled Agents Example

1 The Scenario



Agent has access to:



2 Agent Makes a Mistake

Agent generates:

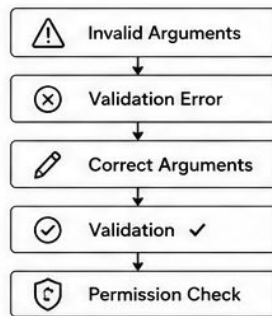
```
start_time = 3 PM  
end_time = 2:30 PM
```

Trace:

Tool selected	✓
Arguments generated	✓
Validation	✗

The system catches the first deviation.

3 Correct & Continue



Now the agent can continue safely.

4 Tool Result Creates a New Problem

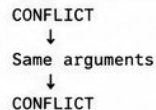
Calendar tool returns:

CONFLICT

Agent attempts:

Retry same request

Trace:



Diagnosis:

Retry strategy is ineffective.



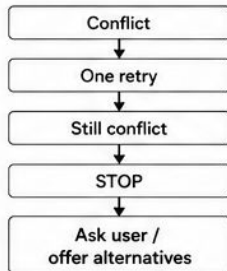
End to End Controlled Agents Example

5 Control the Loop

Add:

Maximum retries = 1

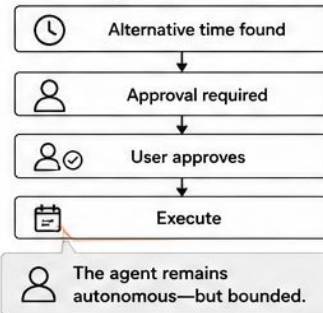
Now:



6 Human Approval

Suppose the user wants the agent to book an alternative time.

Before changing the meeting:



7 Successful Completion

Tool returns:

```
{  "event_id": "evt_123",  "status": "created"}
```

The agent verifies:

Tool success	✓
Event ID	✓
Time	✓
Attendee	✓

Then:

“Done — the meeting has been scheduled.”

8 The Complete Controlled Run



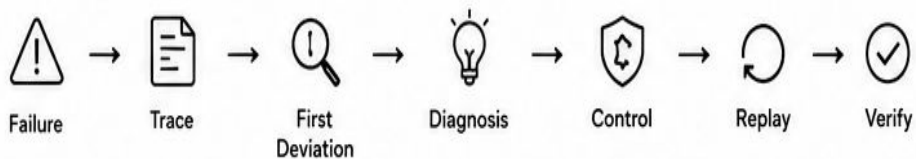
And throughout:

Trace everything important.



End to End Controlled Agents Example

9 From Failure to Control



This is the core workflow of reliable agent development.

10

Q1: What should you inspect first?



The execution trace

Final Self-Check

Q2: What is the likely failure?



Retry/loop handling

Q3: What control could contain it?



Retry limit + escalation



Summary

- **Complex Debugging:** Dynamic decision-making and branching paths require tracing observable steps (*Observe* → *Decide* → *Act* → *Update State*) rather than guessing hidden LLM reasoning.
- **Target the First Deviation:** Early errors compound rapidly; focus troubleshooting on the initial point of failure rather than the final incorrect output.
- **Layer-Specific Fixes:** Match controls directly to the failure source, whether in instructions, tool calls, context, coordination, or verification.
- **Bounded Autonomy:** Enforce safety through risk-tiered approvals (Low = Auto, Med = Confirm, High = Human Approval) enforced at the system level, not just in prompts.
- **Deterministic Guardrails:** Prevent runaways using explicit stopping criteria (cost, turn limits, policies) and rely on external verification instead of LLM self-confidence.
- **The Core Framework:** Always maintain reliability via **Observe** → **Diagnose** → **Control** → **Verify**.



WeCloudData

Thank You

We'd love to hear from you. If you have any questions,
feel free to reach out to our team anytime!